

Université Hassan II de Casablanca

Faculté des Sciences Aïn Chock

Module M11 - Automatisation des tests et qualité logicielle

Rapport final

Rapport final

Automatisation des tests UI, API, performance et sécurité - intégration CI/CD

Réalisé par : [NOM PRÉNOM / MEMBRES]

Filière : IIIA

Encadrant : [NOM DU PROF]

Année universitaire : 2025-2026

Dépôt Git : github.com/essalhi-moussa/Automatisation_des_Tests_et_Qualite_Logicielle

Table des matières

1. Introduction	3
2. Architecture du projet	3
3. Scénarios de test	3
4. Scripts : extraits significatifs	
5. Résultats	5
5.1 Tests UI (Selenium, Allure)	5
5.2 Tests API (RestAssured et Postman / Newman)	5
5.3 Performance (JMeter)	6
5.4 Sécurité (OWASP ZAP)	7
6. Pipeline CI/CD	8
7. Difficultés rencontrées et solutions	
8. Recommandations d'amélioration	9
9. Conclusion	9

1. Introduction

L'équipe QA de l'application web de gestion d'étudiants a automatisé les tests d'interface, d'API, de performance et de sécurité, puis les a intégrés dans un pipeline CI/CD. Ce rapport présente l'architecture retenue, les scénarios, les extraits de code significatifs, les résultats réels des exécutions, les difficultés rencontrées et les pistes d'amélioration.

Les plateformes imposées sont utilisées telles quelles : Formy pour l'interface, Reqres pour l'API, JMeter pour la charge, OWASP ZAP pour la sécurité et GitLab CI pour l'intégration continue (Jenkinsfile en alternative). Tous les chiffres ci-dessous proviennent des rapports archivés dans `reports/` (fichier de synthèse `reports/summary.json`).

2. Architecture du projet

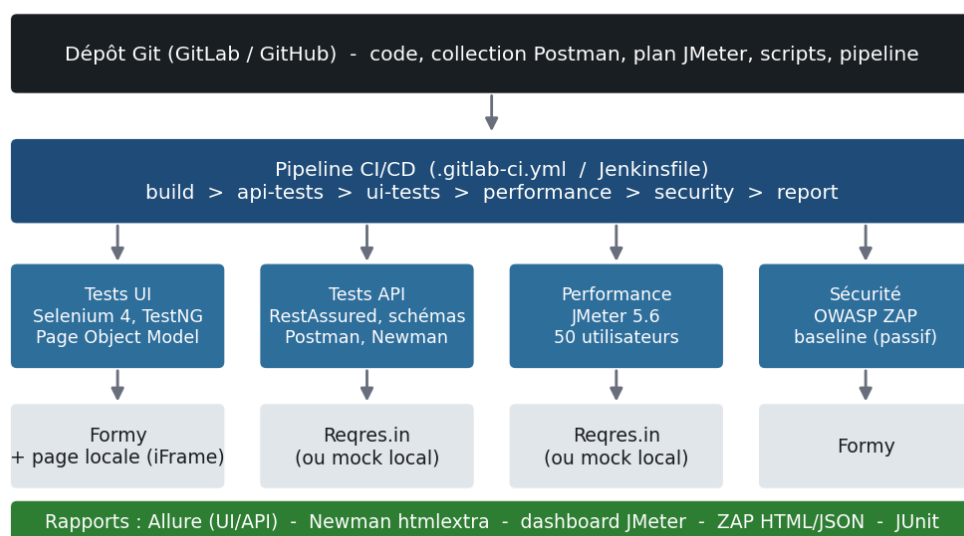


Figure 1 - Architecture de la solution de test

Tableau 1 - Structure du dépôt

Dossier	Contenu
src/test/java/.../base, listeners, config	BaseTest (WebDriver en ThreadLocal), DriverFactory (Selenium Manager), TestListener (capture Allure), Config
src/test/java/.../pages	Page Objects : FormPage, ModalPage, SwitchWindowPage, DropdownPage, RadioButtonPage, ButtonsPage, CheckboxPage, EnabledPage, FramesPage
src/test/java/.../tests/ui et api	Tests Selenium (21 dans la suite ui) et tests RestAssured (9 dans la suite api)
src/test/resources	config.properties, schémas JSON, suites TestNG (smoke, ui, api, external, all)
postman/, jmeter/, mock/, scripts/	Collection v2.1, plan de charge et CSV, mock Reqres, scripts ZAP / JMeter / Newman
.gitlab-ci.yml, Jenkinsfile, reports/, docs/	Pipelines, rapports d'exécution, plan de tests et rapport

3. Scénarios de test

15 scénarios UI, 8 scénarios API, 2 de performance et 2 de sécurité sont décrits en détail (préconditions, étapes, données, résultat attendu, critère, priorité) dans le plan de tests. Le tableau suivant en donne la vue d'ensemble.

Tableau 2 - Vue d'ensemble des scénarios

ID	Scénario	Prio.	ID	Scénario	Prio.
UI-01	Inscription valide (formulaire complet)	Haute	UI-15	iFrame et iFrame imbriqué (page locale déterministe)	Haute

ID	Scénario	Prio.	ID	Scénario	Prio.
UI-02	Soumission avec champs vides	Moyenne	API-01	GET /api/users?page=2	Haute
UI-03	Liste déroulante « années d'expérience »	Moyenne	API-02	GET /api/users/2	Haute
UI-04	Ouverture et fermeture du modal (Close)	Haute	API-03	GET /api/users/23 (inexistant)	Haute
UI-05	Bouton OK du modal	Basse	API-04	POST /api/users	Haute
UI-06	Alerte JavaScript	Haute	API-05	PUT /api/users/2	Moyenne
UI-07	Nouvel onglet	Moyenne	API-06	DELETE /api/users/2	Moyenne
UI-08	Navigation par le menu déroulant	Moyenne	API-07	POST /api/login valide	Moyenne
UI-09	Boutons radio exclusifs	Haute	API-08	POST /api/login sans mot de passe	Haute
UI-10	Cases à cocher indépendantes	Moyenne	PERF-01	Charge nominale : 50 utilisateurs, GET et POST	Haute
UI-11	Boutons et groupe de boutons	Moyenne	PERF-02	Comportement face à la limitation de débit (HTTP 429)	Haute
UI-12	Champ actif / champ désactivé	Basse	SEC-01	Analyse passive ZAP de Formy (baseline)	Haute
UI-13	Frames imbriqués (site tiers)	Moyenne	SEC-02	Recherche de réflexion XSS sur /form et /thanks	Moyenne
UI-14	Frame du bas puis retour au contenu principal (site tiers)	Basse			

Choix liés aux plateformes. Formy n'a pas de page de connexion : la page /form joue le rôle du formulaire d'inscription. Formy n'a pas d'iFrame : l'exigence est couverte par une page locale autonome (UI-15) et par le site the-internet.herokuapp.com (UI-13, UI-14), isolé dans le groupe `external` car ses chargements dépassent parfois 30 secondes.

4. Scripts : extraits significatifs

```
private static final ThreadLocal<WebDriver> DRIVER = new ThreadLocal<>();

public static WebDriver getDriver() {
    return DRIVER.get();
}

@BeforeMethod(alwaysRun = true)
public void setUp() {
    DRIVER.set(DriverFactory.create(pageLoadStrategy()));
}

@AfterMethod(alwaysRun = true)
public void tearDown() {
    WebDriver driver = DRIVER.get();
    if (driver != null) {
        driver.quit();
        DRIVER.remove();
    }
}
```

Extrait 1 - BaseTest : un WebDriver par thread

```
@Step("Soumettre le formulaire")
public ThanksPage submit() {
    WebElement btn = visible(SUBMIT);
    scrollIntoView(btn);
    click(SUBMIT);
    wait.until(ExpectedConditions.textToBePresentInElementLocated(By.tagName("h1"), "Thanks"));
    return new ThanksPage(driver);
}
```

Extrait 2 - Page Object : attente explicite du contenu de /thanks (et non de l'URL seule)

```

public void listerLesUtilisateurs() {
    Response r = send(() -> given().spec(spec()).queryParam("page", 2).when().get("/api/users"));
    r.then()
        .statusCode(200)
        .time(lessThan(MAX_TIME_MS))
        .body("page", equalTo(2))
        .body("per_page", equalTo(6))
        .body("data", hasSize(6))
        .body("data[0].id", equalTo(7))
        .body("data.email", everyItem(containsString("@")))
        .body(matchesJsonSchemaInClasspath(schema("users-list")));
}

```

Extrait 3 - Test RestAssured : statut, corps, schéma JSON et temps

```

pm.test('Statut HTTP 201', function () { pm.response.to.have.status(201); });
pm.test('Temps de réponse < ' + pm.environment.get('maxResponseTime') + ' ms', function () {
    pm.expect(pm.response.responseTime).to.be.below(Number(pm.environment.get('maxResponseTime')));
});
pm.test('Schéma JSON valide', function () { pm.response.to.have.jsonSchema(schema); });

```

Extrait 4 - Assertions Postman (statut, temps, schéma), variables d'environnement baseUrl / apiKey / maxResponseTime

5. Résultats

5.1 Tests UI (Selenium, Allure)

La suite `ui` (profil Maven `ui`, Chrome headless, 3 classes en parallèle) donne **21 tests réussis sur 21** en 32,9 s. Le rapport Allure complet est dans `reports/ui/allure-local/index.html`.

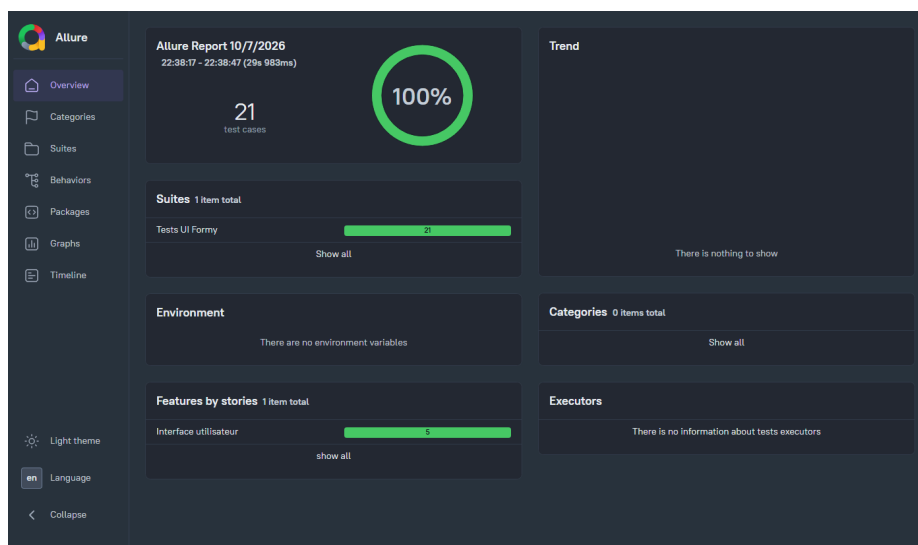


Figure 2 - Rapport Allure de la suite UI (capture réelle)

Le groupe `external` (`the-internet.herokuapp.com/nested_frames`) a été exécuté séparément : **3 réussis, 1 échec définitif, 4 relances** (relance automatique limitée à 2). Ce site tiers répond de façon irrégulière ; ce résultat est rapporté tel quel et n'entre pas dans le critère de sortie de la suite `ui`.

5.2 Tests API (RestAssured et Postman / Newman)

Tableau 3 - Résultats des tests API

Exécution	Réussis	Durée
RestAssured - API publique	9 / 9	10,1 s
RestAssured - mock local	9 / 9	9,4 s
Newman - API publique (8 requêtes)	37 / 37 assertions	2037 ms (moy. réponse 151 ms)
Newman - mock local (8 requêtes)	37 / 37 assertions	946 ms (moy. réponse 13 ms)

Chaque test vérifie le code HTTP, le contenu JSON, le schéma (fichiers `src/test/resources/schemas`) et le temps de réponse (seuil de 3 s). Les cas négatifs couvrent GET `/users/23` (404) et POST `/login` sans mot de passe (400). Comportements constatés : POST et PUT ne persistent rien, DELETE renvoie 204 sans corps, et l'identifiant renvoyé par POST est une chaîne.

Constat important : Reqres limite fortement le débit. Les réponses portent les en-têtes `Ratelimit-Limit: 20` (20 requêtes par minute) et `X-Ratelimit-Limit: 40` (quota journalier). Nous avons épuisé ce quota pendant la mise au point : l'API répond alors 429 jusqu'à la réinitialisation à minuit UTC. Les tests rejouent donc les requêtes après un 429 (attente du délai `Ratelimit-Reset`) et un mock local fidèle au contrat permet de poursuivre sans quota.

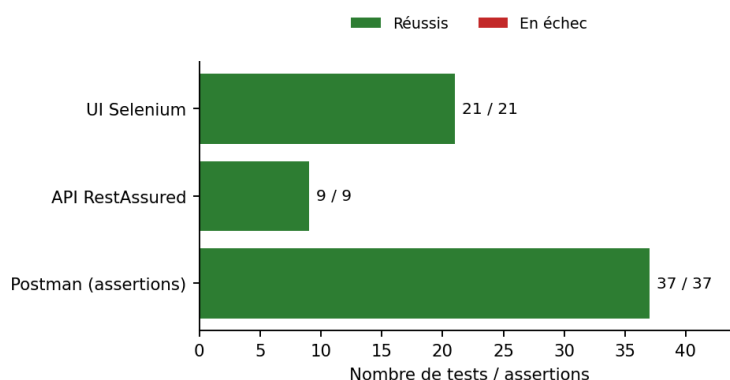


Figure 3 - Synthèse des tests UI et API

5.3 Performance (JMeter)

Plan : 50 utilisateurs, montée en charge de 10 s, 10 boucles par utilisateur, temps de réflexion aléatoire de 0,5 à 1,5 s, GET `/api/users?page=2` puis POST `/api/users` alimenté par un CSV, avec assertions (code, JSONPath, durée < 3 s). Exécution en mode CLI avec génération du tableau de bord HTML.

Tableau 4 - Résultats JMeter

Cible	Requête	Échant.	Erreurs	Moy. (ms)	P90	P95	P99	Débit (req/s)
API publique Reqres	GET <code>/api/users?page=2</code>	500	0,00 %	57	114	138	212	10,6
API publique Reqres	POST <code>/api/users</code>	500	95,80 %	84	92	98	120	10,5
API publique Reqres	Total	1000	47,90 %	71	92	125	161	20,1
Mock local	GET <code>/api/users?page=2</code>	500	0,00 %	2	4	6	7	10,7
Mock local	POST <code>/api/users</code>	500	0,00 %	3	4	4	5	10,6
Mock local	Total	1000	0,00 %	2	4	4	7	20,2

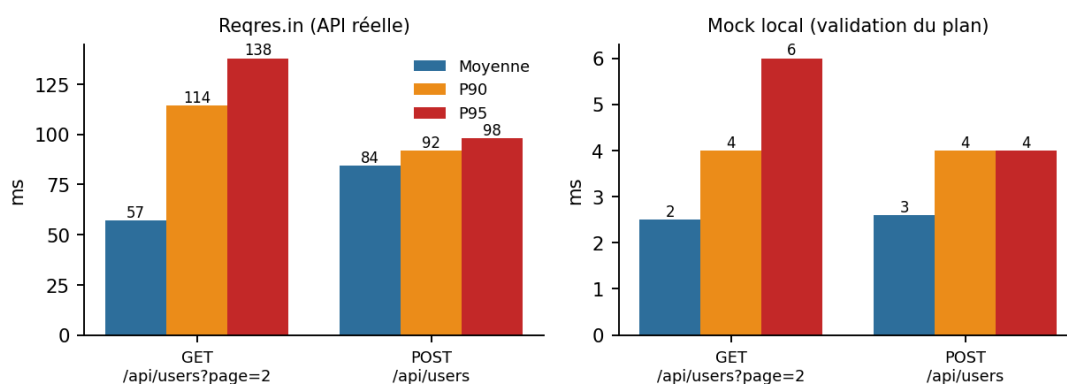


Figure 4 - Temps de réponse (moyenne, P90, P95)

API publique. Les 1000 échantillons donnent 47,9 % d'erreurs au total, mais la répartition est très contrastée. Les 500 GET ont tous réussi (200 : 500), avec une moyenne de 57 ms, un P90 de 114 ms et un P95 de 138 ms. Côté POST, seules 21 requêtes ont abouti (201) ; les 479 suivantes ont reçu un **HTTP 429**, le premier 6 s après le

démarrage. Ce chiffre correspond au reliquat du quota journalier de 40 requêtes, déjà entamé par nos tests fonctionnels. Les GET portent des en-têtes de cache public (Cache-Control: public, s-maxage=60, servis par Cloudflare) : un service depuis le cache du CDN est l'explication la plus probable de leur succès, mais elle n'est pas démontrée (les en-têtes de réponse n'ont pas été enregistrés). Conclusion : l'API publique ne permet pas de mesurer une capacité ; le débit global de 20,1 req/s est imposé par les temps de réflexion et les 429 mesurent la politique de limitation, pas une saturation du serveur. **Mock local.** Le même plan, sans limitation de débit, s'exécute sans erreur : 1000 échantillons, 0,0 % d'erreurs, moyenne 2,5 ms, P90 4 ms, P95 4 ms, débit 20,2 req/s. Ces valeurs valident le script et l'outillage ; elles ne disent rien des performances de Reqres puisque le serveur est local.

5.4 Sécurité (OWASP ZAP)

Analyse baseline (spider puis analyse passive) de <https://formy-project.herokuapp.com> avec ZAP 2.17.0 : **14 types d'alertes** (élevé 0, moyen 4, faible 6, informatif 4). Aucune attaque active n'a été lancée sur ce site tiers.

Tableau 5 - Alertes principales, correspondance OWASP / CWE et correctifs

Alerte ZAP	Risque	OWASP Top 10 (2021)	CWE	Recommandation
Absence of Anti-CSRF Tokens	Moyen (confiance faible)	A01 Contrôle d'accès défaillant	352	Ajouter un jeton anti-CSRF aux formulaires POST (/fileupload).
Content Security Policy (CSP) Header Not Set	Moyen (confiance forte)	A05 Mauvaise configuration	693	Définir Content-Security-Policy: default-src 'self' avec les sources tierces strictement nécessaires.
Sub Resource Integrity Attribute Missing	Moyen (confiance forte)	A08 Intégrité des logiciels et des données	345	Ajouter integrity="sha384-..." et crossorigin aux scripts et feuilles de style externes.
Vulnerable JS Library	Moyen (confiance moyenne)	A06 Composants vulnérables ou obsolètes	1395	Mettre jQuery à jour (3.5.0 ou plus ; CVE-2020-11022/11023, CVE-2019-11358).
Cookie Without Secure Flag	Faible (confiance moyenne)	A05 Mauvaise configuration	614	Positionner Secure sur le cookie de session.
Cookie without SameSite Attribute	Faible (confiance moyenne)	A01 Contrôle d'accès défaillant	1275	Positionner SameSite=Lax (ou Strict).
Cross-Domain JavaScript Source File Inclusion	Faible (confiance moyenne)	A08 Intégrité des logiciels et des données	829	Héberger localement les bibliothèques ou les verrouiller par SRI.
Strict-Transport-Security Header Not Set	Faible (confiance forte)	A02 Défaillances cryptographiques	319	Envoyer Strict-Transport-Security: max-age=31536000; includeSubDomains.
Timestamp Disclosure - Unix	Faible (confiance faible)	A05 Mauvaise configuration	497	Pas d'action requise (faux positif probable : valeurs numériques).
X-Content-Type-Options Header Missing	Faible (confiance moyenne)	A05 Mauvaise configuration	693	Envoyer X-Content-Type-Options: nosniff sur toutes les ressources, y compris statiques.

Contrôles complémentaires : une recherche de réflexion XSS (marqueur inoffensif dans les paramètres de /form et /thanks) n'a produit **aucune réflexion** ; le formulaire Formy n'envoie d'ailleurs pas ses champs au serveur (le bouton Submit est un lien vers /thanks). Points positifs relevés sur les en-têtes : X-Frame-Options: SAMEORIGIN, X-Content-Type-Options: nosniff sur les pages, cookie de session HttpOnly. Une clé d'API Google Maps est visible dans le code source de /autocomplete : elle doit être restreinte par référent HTTP.

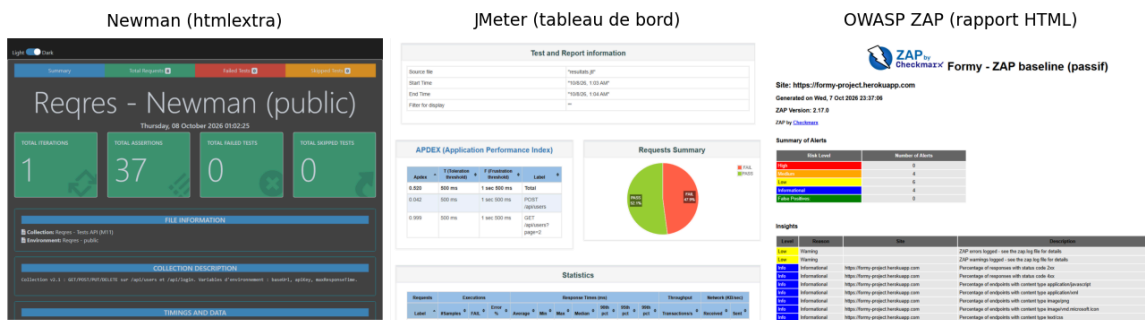


Figure 5 - Captures réelles des rapports Newman, JMeter et ZAP (fichiers complets dans reports/)

6. Pipeline CI/CD

Notifications : e-mail GitLab (pipeline status) + webhook Slack/Discord en fin de pipeline

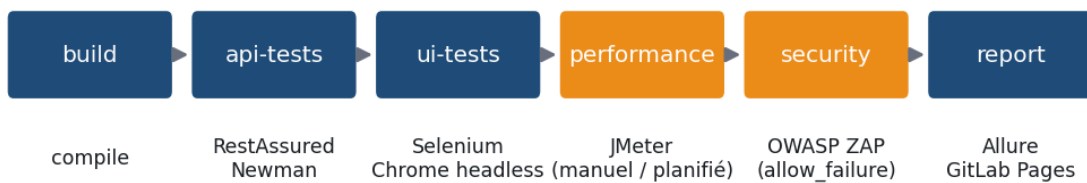


Figure 6 - Étapes du pipeline GitLab CI

- **build** : compilation Maven ; cache du dépôt Maven (clé = hash du pom.xml).
- **api-tests** : RestAssured (rapport JUnit exposé à GitLab) et Newman (HTML htmlextra + JUnit).
- **ui-tests** : Selenium sur le service `selenium/standalone-chrome` (navigateur distant via `remote.url`).
- **performance** : JMeter, déclenchement manuel ou planifié (pour ne pas épuiser le quota Reqres à chaque commit).
- **security** : ZAP baseline (image officielle), `allow_failure: true`.
- **report** : génération du rapport Allure, publié avec GitLab Pages ; artefacts conservés 30 jours.
- **Notifications** : e-mails de statut de pipeline (Settings > Integrations > Pipeline status emails) et webhook Slack/Discord via les variables CI `SLACK_WEBHOOK_URL` / `DISCORD_WEBHOOK_URL`.
- **Jenkinsfile** équivalent : étapes parallèles API, Allure Plugin, JUnit, `emailext`.

Statut de vérification. Les fichiers `.gitlab-ci.yml` et `Jenkinsfile` ont été validés syntaxiquement et chacune de leurs commandes a été exécutée localement (Maven, Newman, JMeter, ZAP, génération Allure). Le pipeline n'a pas encore tourné sur un runner GitLab : les captures ci-dessous sont à insérer après le premier push.

Tableau 6 - Captures GitLab à ajouter

Capture à insérer	Où la prendre
[Capture 1 - Pipeline réussi : graphe des 6 stages]	GitLab > CI/CD > Pipelines
[Capture 2 - Rapport Allure publié sur GitLab Pages]	Settings > Pages (URL du projet)
[Capture 3 - Onglet Tests (rapport JUnit) d'un pipeline]	Pipeline > Tests
[Capture 4 - E-mail ou message Slack de fin de pipeline]	Boîte de réception / canal

7. Difficultés rencontrées et solutions

Tableau 7 - Problèmes et solutions

Difficulté	Solution
Formy n'a pas de page de connexion ni d'iFrame	Page /form utilisée comme inscription ; page locale data: avec iFrames imbriqués, et nested_frames en complément
Quota Reqres de 20 req/min et 40 req/jour : réponses 429	Gestionnaire de 429 (attente de RateLimit-Reset), mock local fidèle, 429 analysé comme résultat de performance
Page /thanks : l'URL change avant le contenu (test intermittent)	Attente explicite du titre « Thanks » au lieu de l'URL
Bouton OK du modal sans effet	Le test décrit le comportement réel et ferme le modal avec Close
the-internet : chargement > 30 s, éditeur TinyMCE en lecture seule	Abandon de /iframe, frames imbriqués avec stratégie de chargement NONE, groupe external isolé
Pas de Docker sur le poste	ZAP local avec plan Automation Framework équivalent ; image Docker conservée pour la CI
Échappements JSON des schémas (shell Windows)	Expressions régulières simplifiées ([0-9], [..]) et validation JSON de chaque schéma

8. Recommandations d'amélioration

- **Qualité des tests** : ajouter l'historique Allure (tendances), exécuter Firefox et Edge en matrice, paralléliser sur un Selenium Grid.
- **API** : tests de contrat (Pact), virtualisation de Reqres (WireMock) pour la CI afin de ne dépendre d'aucun quota, clé API personnelle stockée en variable CI masquée.
- **Performance** : définir des seuils bloquants (P95, taux d'erreur) dans le pipeline, tester sur un environnement dédié et non sur une API publique limitée, enregistrer les en-têtes de réponse (dont cf-cache-status) pour distinguer les réponses servies par le cache.
- **Sécurité** : appliquer les en-têtes CSP, HSTS, cookies Secure et SameSite, mettre à jour jQuery, ajouter SRI ; ajouter un scan de dépendances et un scan ZAP authentifié sur l'application réelle.
- **Processus** : exécuter le pipeline à chaque merge request, notifier uniquement les échecs, tenir la matrice de traçabilité à jour.

9. Conclusion

Le projet livre une chaîne de tests automatisés complète et reproductible : 21 tests UI Selenium réussis sur 21, 9 tests API RestAssured réussis sur 9, 37 assertions Postman validées sur 37, un plan de charge JMeter de 50 utilisateurs et un scan ZAP assorti de recommandations chiffrées. Le principal enseignement est que la qualité des résultats dépend autant de la cible que des scripts : les limites de Reqres (429) et l'instabilité de certains sites de démonstration imposent de séparer les tests déterministes (mock, page locale) des tests sur services tiers, et de rapporter honnêtement les écarts plutôt que de les masquer. Il reste à exécuter le pipeline sur un runner GitLab, à y insérer les captures prévues et à enrichir le plan selon les recommandations du chapitre 8.